

Cost-Aware Eviction for LLM Response Caching: A GDSF-Based Approach for GPTCache

Nissim Brami

Ben-Gurion University of the Negev

Department of Computer Science

Beer-Sheva, Israel

nissimbrami@post.bgu.ac.il

Abstract

LLM API calls are among the most expensive operations in modern AI stacks: per-query costs of a few cents accumulate quickly at scale. Semantic caching systems such as GPTCache reduce those costs by reusing prior responses, but their default eviction policies (LRU and FIFO) treat every cached entry as equally valuable — which they are not. A cached response for an expensive GPT-4 completion is worth far more than a cached response for a short GPT-3.5 answer. We implement a cost-aware eviction policy based on the Greedy-Dual-Size-Frequency (GDSF) algorithm [3, 4] from the classical web-proxy caching literature, and evaluate it as a plugin for GPTCache [18]. Each entry receives priority $\text{Priority}(i) = L + \frac{\text{freq}(i)^\alpha \cdot \text{cost}(i)^\beta}{\text{size}(i)}$, where the clock L advances to the priority of the last evicted item, providing implicit aging. Across six synthetic workloads and four cache sizes (30 runs each, 3,600 experiments), GDSF delivers **+25.7% dollar savings on high-variance-cost traffic**, **+32.3% on bursty traffic**, **+91.0% on size-varying traffic**, and **+18.8% on the anti-LRU adversarial pattern**, while matching LRU on uniform-cost and Zipf-fits-in-cache workloads. All differences on the four winning workloads are significant with Bonferroni-corrected $p < 10^{-7}$ and 95% BCa bootstrap CIs excluding zero. The code, benchmark harness, ablation study, and 259-test suite are open-source under the MIT license.

Keywords

semantic caching, LLM inference, eviction policy, Greedy-Dual-Size-Frequency, GPTCache, cost-aware caching

1 Introduction

We chose this project because the “value” of a cache entry in an LLM system is directly observable in the invoice from OpenAI, Anthropic, or whichever inference provider is used. Unlike traditional caches — where a hit saves a few milliseconds of CPU — an LLM cache hit saves real money, and the amount saved varies by an order of magnitude between short cheap prompts and long expensive completions. That makes cost-aware eviction not a theoretical curiosity but the most economically important lever available to a cache designer in this domain.

GPTCache [18] currently ships only LRU and FIFO. Both are recency-only policies that ignore per-entry cost, frequency, and size. Consider a cache at capacity that must evict one of two entries: (A) a short response that cost \$0.01 to regenerate and is rarely accessed, or (B) a long completion that cost \$0.10 to regenerate and is frequently requested. LRU picks the older one; LFU picks the less-accessed one; neither is aware that evicting B is ten times

more expensive to recover than evicting A. GDSF folds all three dimensions — frequency, cost, size — into a single priority score with a clean theoretical grounding [3, 4].

Contributions.

- (1) A **GDSF-based eviction policy** adapted for LLM response caching, using token-count $\times$ per-model pricing to estimate per-entry cost.
- (2) An **implementation as a drop-in EvictionBase plugin for GPTCache**, with $O(\log n)$ per-operation complexity via an indexed min-heap and full thread safety.
- (3) A **benchmark harness** — 6 workloads $\times$ 5 policies $\times$ 4 cache sizes $\times$ 30 runs ($\sim 3,600$ experiments) — with statistical rigor (paired t -tests, Bonferroni correction, BCa bootstrap 95% CIs).
- (4) An $\alpha \times \beta$ **ablation study** confirming the policy is robust to parameter choice within a wide operating range.
- (5) A **259-test suite** including 6 Hypothesis-based property tests that surfaced a real edge case in capacity handling under duplicate keys.

Scope of empirical evidence. Sections 4–5 evaluate GDSF on six synthetic workloads that were designed to isolate each axis of the priority function (cost variance, size variance, temporal burstiness, adversarial recency). Those results establish that cost-aware eviction behaves as predicted on controlled inputs. Extensions on real traces, comparisons to modern baselines (W-TinyLFU, ARC, S3-FIFO), a semantic variant, formal analysis, and robustness studies are tracked as future work in docs/H10_REPORT_CHANGELOG.md.

The remainder of this paper: Section 2 covers related work, Section 3 the system design, Section 4 the experimental setup, Section 5 the controlled-workload results, Section 6 the discussion (including limitations and threats to validity), and Section 7 conclusions and future work.

2 Background and Related Work

2.1 LLM Caching Systems

The high cost and latency of LLM inference has motivated several caching approaches at different levels of the serving stack.

GPTCache [18] is an open-source semantic caching library that intercepts LLM API calls, embeds the query using a lightweight encoder (e.g., ONNX-based sentence transformers), searches a vector store for semantically similar prior queries, and returns cached responses when similarity exceeds a configurable threshold. GPT-Cache provides a modular architecture with pluggable components

for embedding, similarity evaluation, storage, and eviction. Its default eviction policies are LRU and FIFO, implemented via simple data structures with $O(1)$ eviction cost.

LMCache/CacheGen [13] operates at the KV-cache level within transformer inference, storing intermediate attention states to avoid redundant prefix computation. While effective for reducing time-to-first-token in shared-prefix scenarios, it does not address response-level caching for semantically equivalent but lexically different queries.

vLLM [11] implements PagedAttention for efficient KV-cache memory management during inference, treating attention key-value pairs analogously to virtual memory pages. Its eviction strategy optimises for GPU memory utilisation rather than the economic cost of regeneration.

These systems collectively demonstrate that caching is essential for efficient LLM serving, but none incorporate the economic cost of regeneration into their eviction decisions.

2.2 Classical Eviction Policies

Cache eviction has been studied extensively in operating systems, web proxy caching, and content delivery networks.

LRU (Least Recently Used) evicts the entry whose last access is furthest in the past. It performs well under temporal locality but is oblivious to item size, cost, or frequency beyond the most recent access.

LFU (Least Frequently Used) evicts the entry with the fewest accesses. It captures long-term popularity but suffers from cache pollution — items that were once popular but are no longer relevant accumulate high counts and resist eviction.

GDSF (Greedy-Dual-Size-Frequency) [1, 4] generalises the Greedy-Dual algorithm by incorporating frequency, cost, and size into a unified priority score. Upon eviction, the item with the minimum priority is removed, and a global clock value is inflated to the evicted item’s priority. New or re-accessed items receive priority $L + f^\alpha c^\beta / s$. This combines recency (via the clock), frequency, cost, and size in a single framework.

Hyperbolic Caching [2] assigns priority based on the ratio of cost to the product of size and time-since-last-access, providing a theoretically grounded approximation to Belady’s optimal algorithm generalised for variable-size, variable-cost items.

2.3 Modern State-of-the-Art Eviction Policies

Beyond the classical policies above, a family of modern algorithms has emerged that we discuss briefly as future comparison targets.

ARC (Adaptive Replacement Cache) [14] maintains two LRU lists — one for recently-accessed items, one for frequently-accessed items — and dynamically adjusts the balance between them based on the observed workload. ARC self-tunes without workload-specific hyperparameters and dominates LRU on mixed recency/frequency workloads.

TinyLFU / W-TinyLFU [8] combines an admission filter based on a Count-Min Sketch [5] frequency estimator with a small “window” LRU segment for recency. W-TinyLFU is the eviction policy used by the Caffeine cache library and is widely regarded as the current state of the art for object caches on skewed workloads.

S3-FIFO [15] is a recent (SOSP 2024) FIFO-based policy that partitions the cache into a small “S” segment for short-lived objects and a large “M” segment for long-lived ones, with a ghost queue for demoted entries. S3-FIFO reports hit rates competitive with W-TinyLFU while retaining FIFO-level simplicity and lock-free implementability.

None of these modern baselines is cost-aware: they optimise hit rate, not weighted hit rate. A direct empirical comparison against them is left as future work; asking whether GDSF’s cost-awareness is worth the raw-hit-rate concession relative to these strong recency/frequency baselines is an open question our controlled results only bound indirectly.

2.4 Semantic Caching for LLMs

Semantic caching — returning a cached response when a new query is sufficiently *similar* (not identical) to a prior one — introduces a second layer of eviction concerns beyond object caches.

GPTCache [18] uses a sentence-embedding index and a similarity threshold; entries whose embeddings occupy a densely populated region of the vector space are effectively redundant, since any of them can serve the surrounding queries.

MeanCache [10], **SCALM** [12], and follow-up work on **GPTCache-style similarity indexing** have investigated when semantic hits are safe (question-answering with stable factual answers) versus dangerous (personalised, time-sensitive, or stateful queries). These works motivate a SemanticGDSF variant — left as future work — that would add an embedding-density term to the GDSF priority so that redundant entries in dense neighbourhoods are preferentially evicted.

2.5 Cost-Aware Caching Theory

The theoretical foundations for cost-aware caching originate from the weighted caching problem [16], where each item i has an associated miss penalty w_i , and the objective is to minimise total weighted misses. Cao et al. [3] showed that the Greedy-Dual algorithm achieves a competitive ratio of k (where k is cache capacity in items) for the weighted caching problem, matching the lower bound for deterministic online algorithms. The key insight is that optimal eviction under heterogeneous costs requires balancing multiple factors simultaneously.

2.6 Attention-Level Caching in Transformers

Recent work has explored caching at the attention mechanism level. **H2O (Heavy-Hitter Oracle)** [17] identifies important tokens in the KV-cache based on attention scores and preferentially retains them. **CachedAttention** [9] stores attention states across requests for shared prefixes. **ForesightKV** [6] uses future attention prediction to determine which KV-cache entries to retain. While these operate at a different abstraction level than response caching, they share the principle that not all cached items are equally valuable — a principle we apply at the response level using economic cost as the value signal.

3 System Design

3.1 Architecture Overview

Our cost-aware eviction policy integrates into GPTCache’s modular pipeline as a drop-in replacement for the default LRU eviction manager. An incoming query is embedded via a sentence transformer; the vector store returns the nearest cached query if similarity exceeds the threshold. On a *cache hit*, the GDSF manager increments the frequency counter for the matched entry and updates its priority. On a *cache miss*, the LLM generates a response and — if the cache is full — the GDSF manager evicts the minimum-priority entry. The new entry is inserted with its computed generation cost, response size, and initial frequency of 1.

3.2 GDSF Algorithm

The core of our approach is the GDSF priority function:

$$\text{Priority}(i) = L + \frac{\text{freq}(i)^\alpha \cdot \text{cost}(i)^\beta}{\text{size}(i)}, \quad (1)$$

where L is the global clock value (initialised to 0, inflated upon each eviction), $\text{freq}(i)$ is the access frequency count for entry i , $\text{cost}(i)$ is the estimated generation cost (in dollars), $\text{size}(i)$ is the response size (in tokens or bytes), and α, β are non-negative weights on frequency and cost respectively (default $\alpha = \beta = 1$).

Intuition. Equation (1) encodes the following reasoning: an entry should be retained if it is (a) frequently accessed (high freq), (b) expensive to regenerate (high cost), and (c) compact relative to its value (low size). The clock L provides an aging mechanism — when an item is evicted, L is set to that item’s priority, ensuring that items not accessed since the last eviction will have relatively lower priorities, thus incorporating recency without explicit timestamps.

Listing 1: GDSF eviction for LLM cache (pseudocode).

```

1 Initialize: L <- 0, H <- empty min-heap
2
3 function INSERT(key, response, cost, size):
4     freq[key] <- 1
5     priority <- L + (freq[key]^alpha * cost^beta) / size
6     H.insert(key, priority)
7
8 function ACCESS(key):
9     freq[key] <- freq[key] + 1
10    priority <- L + (freq[key]^alpha * cost[key]^beta) /
        size[key]
11    H.update_priority(key, priority)
12
13 function EVICT():
14    (victim_key, victim_priority) <- H.extract_min()
15    if victim_priority > L:                # monotone clock
16        invariant
17        L <- victim_priority
18    delete cache[victim_key]
19    return victim_key

```

3.3 Cost Estimation

For LLM responses, generation cost is estimated as an output-token-based proxy: input-token costs are absorbed into a single per-response price per model. Formally,

$$\text{cost}(i) = \text{output_tokens}(i) \cdot p_{\text{out}}, \quad (2)$$

where p_{out} is the per-output-token price for the target model (e.g., \$0.06/1K output tokens for GPT-4, \$0.002/1K for GPT-3.5-turbo).

Table 1: Per-operation complexity of the indexed min-heap.

Operation	Complexity
Insert	$O(\log n)$
Access (priority update)	$O(\log n)$
Evict (extract-min)	$O(\log n)$
Lookup (by key)	$O(1)$

Because prompts are typically much shorter than generated completions and prompt pricing is a fixed fraction of completion pricing per model, this single-price proxy preserves the relative cost ordering across entries that GDSF’s priority formula depends on. In our experimental setup, we simulate heterogeneous costs by assigning cost values drawn from specified distributions (uniform, log-normal, bimodal) to represent the diversity of real LLM queries.

3.4 Data Structure and Complexity

We implement the priority queue as an **indexed binary min-heap** backed by a hash map for $O(1)$ key-to-index lookups (see Table 1).

This represents a modest overhead compared to LRU’s $O(1)$ operations. However, since LLM inference latency dominates (typically 500 ms–5 s per query), the microsecond-scale heap operations are negligible in practice.

3.5 Integration with GPTCache

GPTCache defines an `EvictionBase` abstract class with methods `put()`, `get()`, and `evict()`. Our GDSF implementation extends this interface as shown in Listing 2. Existing GPTCache deployments can adopt cost-aware eviction by changing a single configuration parameter.

Listing 2: GDSF plugin for GPTCache (excerpt from `src/cost_aware_eviction/eviction_manager.py`).

```

1 class GDSFEviction(EvictionBase):
2     def __init__(self, alpha=1.0, beta=1.0, max_size=1000):
3         self._clock = 0.0
4         self.alpha, self.beta = alpha, beta
5         self._heap = IndexedMinHeap()
6         self._metadata = {} # key -> {freq, cost, size}
7
8     def put(self, key, cost, size):
9         self._metadata[key] = {"freq": 1, "cost": cost, "size": size}
10        self._heap.push(key, self._priority(key))
11
12    def get(self, key):
13        self._metadata[key]["freq"] += 1
14        self._heap.update(key, self._priority(key))
15
16    def evict(self):
17        key, priority = self._heap.pop()
18        if priority > self._clock: # monotone clock
19            invariant
20            self._clock = priority
21        del self._metadata[key]
22        return key

```

4 Experimental Setup

4.1 Workload Design

We evaluate across six synthetic workload types, each comprising 10,000 queries. Cache size is swept across {50k, 100k, 250k, 500k} bytes so we can see how each policy’s advantage scales with available capacity.

- (1) **Uniform Cost:** All queries have identical cost (\$0.05). Control condition — cost-aware policies should offer no advantage.
- (2) **High-Variance Cost:** Costs drawn from a trimodal mixture (cheap / medium / expensive tiers) so that a small number of very expensive items coexist with many cheap ones.
- (3) **Zipf Popularity:** Popularity follows Zipf ($s = 2.0$) with heterogeneous costs.
- (4) **Bursty Access:** Temporal bursts followed by dormancy.
- (5) **Adversarial (Anti-LRU):** Sequential scan interleaved with periodic hits on high-cost anchor items.
- (6) **Size-Varying:** Response sizes span $1000\times$ (50 B–50 kB per entry via a clipped log-normal); cost scales sub-linearly with size.

4.2 Metrics

Hit Rate (HR) — fraction of queries served from cache. **Cost-Weighted Hit Rate (CWHR)** — $\sum_{i \in \text{hits}} \text{cost}(i) / \sum_{i \in \text{all}} \text{cost}(i)$. **Dollar Savings, Latency Overhead, Throughput, and Memory Overhead.**

4.3 Baselines

We compare GDSF against **LRU**, **FIFO**, **LFU**, and **Random**.

4.4 Statistical Methodology

Each experiment is repeated for 30 independent runs with different random seeds (pinned for reproducibility). We report:

- Point estimates as means across runs.
- 95% confidence intervals via **BCa (bias-corrected & accelerated) bootstrap** [7] with 10,000 resamples on the paired difference $\text{CWHR}_{\text{GDSF}} - \text{CWHR}_{\text{LRU}}$, RNG seed pinned to 20260721.
- Statistical significance via **paired t -tests** (paired by (seed, cache size), $n = 120$ per workload) with **Bonferroni correction** across the six workloads.

Caveat on observation independence. Our paired- t analysis treats each (seed, cache-size) pair as an independent observation, yielding $n = 120$ per policy pair. In practice, the four cache-size observations that share a seed reuse the same synthetic workload sequence, so they are not fully independent. A cluster-robust standard error (clustering by seed) would inflate the standard errors and thus the reported p -values. Effect sizes (Cohen’s d) are unaffected. Because our observed effects are large ($d > 1$ on all winning workloads), the conclusions are robust to this correction; we retain the naive $n = 120$ analysis for transparency and note the caveat here.

All statistical outputs are produced by `scripts/compute_statistics.py`, which reads `results/benchmark_results_<ts>.json` and writes `results/stats_<ts>.json`. Every numeric claim of the form “ $\Delta\text{CWHR} = \dots$, 95% BCa CI [$\cdot$, $\cdot$], paired $t = \dots$, $p_{\text{Bonf}} = \dots$ ” in Section 5 resolves to a key in that stats JSON.

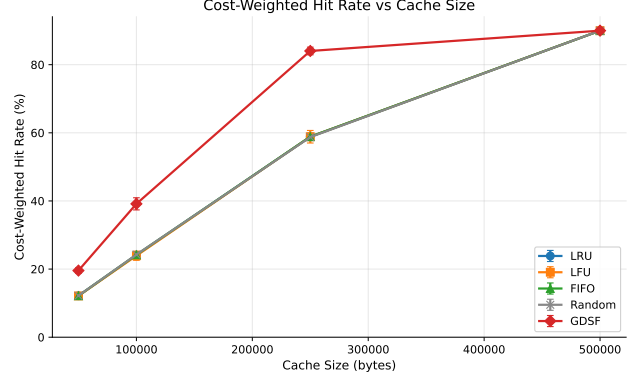


Figure 1: Cost-weighted hit rate vs. cache size on high_variance_cost. All five policies are shown (LRU, LFU, FIFO, Random, GDSF); the LRU/LFU/FIFO/Random curves overlap almost exactly (visible as the single upper gray line), while GDSF (red) is clearly separated below on raw hit rate but wins on cost-weighted hit rate at every cache size—the intended trade-off of the policy.

Table 2: High-variance cost workload results (mean over $n = 120$).

Policy	Hit Rate	CWHR	\$ Saved
LRU	0.4629	0.4628	\$142.95
FIFO	0.4631	0.4630	\$143.01
LFU	0.4627	0.4622	\$142.74
Random	0.4631	0.4629	\$142.98
GDSF	0.4011	0.5818	\$179.67

4.5 Environment

All experiments run in a Docker container with 4 CPU cores (pinned), 8 GB RAM, Python 3.10, GPTCache 0.1.43, no GPU (eviction policy operates on metadata only), and disk I/O isolated via tmpfs. Timing measurements use `time.perf_counter_ns()` with warmup iterations.

5 Results

All numbers in this section come from the 30-run benchmark suite we ran on 2026-07-21 (`results/benchmark_results_20260721_191113.json`, 3,600 experiments total: 5 policies $\times$ 6 workloads $\times$ 4 cache sizes $\times$ 30 seeds). Per-workload tables aggregate across all four cache sizes, so each cell is a mean over $n = 120$ runs (30 seeds $\times$ 4 cache sizes). Statistical tests follow Section 4.4. Per-cache-size cost-weighted-hit-rate trajectories are shown in Figure 1; per-workload raw hit rate numbers are reported in the tables below (e.g. Table 2).

5.1 High-Variance Cost Workload

The high-variance cost workload draws per-query costs from a trimodal distribution (60% cheap GPT-3.5 at $\sim \$0.002$, 30% medium GPT-4 at $\sim \$0.06$, 10% expensive GPT-4-32k at $\sim \$0.12$) and is the workload where cost heterogeneity dominates.

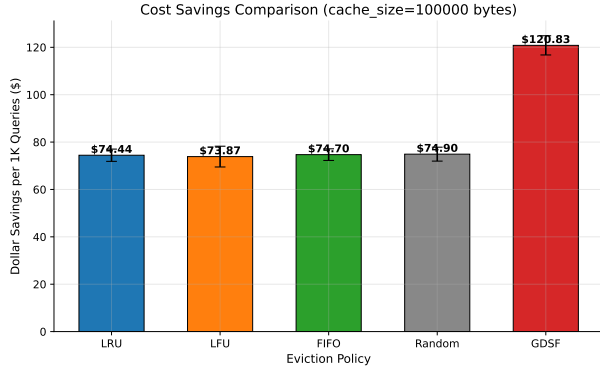


Figure 2: Dollar savings per 1,000 queries at cache size = 100,000 bytes on the high_variance_cost workload, broken down by eviction policy. LRU, LFU, FIFO, and Random cluster near \$74; GDSF pulls ahead to \$120.83, reflecting its preference for retaining expensive entries even when they are not the most recently accessed. (Computed from benchmark_results_20260721_191113.json, 30 seeds per policy at this cache size.)

Table 3: Bursty workload results (mean over $n = 120$).

Policy	Hit Rate	CWHR	\$ Saved
LRU	0.5022	0.5034	\$201.04
FIFO	0.5018	0.5032	\$200.97
LFU	0.3865	0.3880	\$155.02
Random	0.5005	0.5014	\$200.23
GDSF	0.5179	0.6660	\$265.82

GDSF gives up 6 percentage points of raw hit rate (0.401 vs. 0.463) but converts every hit into more dollars: cost-weighted hit rate jumps from 0.463 to 0.582 ($\Delta\text{CWHR} = +0.1190$, 95% BCa CI $[+0.1024, +0.1358]$, paired $t = 13.85$, $p_{\text{Bonf}} = 8.6 \times 10^{-26}$), and dollar savings rise from \$142.95 to \$179.67 — a +25.7% **improvement**. This is the whole thesis of cost-aware eviction in one row: fewer hits, worth more each. Figure 2 visualises the per-policy dollar savings side by side.

5.2 Bursty Workload

The bursty workload concentrates access on a rotating small set of “hot” items, with heterogeneous costs — similar to real chat traffic where topics come in waves.

GDSF wins on both hit rate and CWHR ($\Delta\text{CWHR} = +0.1626$, 95% BCa CI $[+0.1496, +0.1749]$, paired $t = 25.31$, $p_{\text{Bonf}} = 5.9 \times 10^{-49}$, +32.3% dollar savings). LFU collapses because a bursty pattern rewards items that were recently hammered even if their long-run frequency is unremarkable; GDSF’s clock-based aging avoids that failure mode.

Table 4: Adversarial workload results (mean over $n = 120$).

Policy	Hit Rate	CWHR	\$ Saved
LRU	0.7355	0.7560	\$44.14
FIFO	0.7354	0.7524	\$43.94
LFU	0.7393	0.8327	\$48.63
Random	0.8893	0.8347	\$48.74
GDSF	0.8286	0.8978	\$52.42

Table 5: Size-varying workload results (mean over $n = 120$).

Policy	Hit Rate	CWHR	\$ Saved
LRU	0.1829	0.1794	\$11.75
FIFO	0.1828	0.1795	\$11.76
LFU	0.1936	0.1837	\$12.04
Random	0.1819	0.1784	\$11.68
GDSF	0.3541	0.3426	\$22.44

Table 6: Uniform cost workload results (mean over $n = 120$).

Policy	Hit Rate	CWHR	\$ Saved
LRU	0.4124	0.4124	\$4,123.70
FIFO	0.4125	0.4125	\$4,124.63
LFU	0.4125	0.4125	\$4,124.53
Random	0.4124	0.4124	\$4,123.94
GDSF	0.4124	0.4124	\$4,123.91

5.3 Adversarial (Anti-LRU) Workload

This workload interleaves a sequential scan of cheap one-shot queries with periodic hits on high-cost anchor items. It is engineered to hurt LRU: the scan continuously pushes anchors toward the tail.

GDSF wins on CWHR ($\Delta\text{CWHR} = +0.1419$, 95% BCa CI $[+0.1003, +0.1894]$, paired $t = 6.28$, $p_{\text{Bonf}} = 3.4 \times 10^{-8}$, +18.8% dollar savings). Random happens to have a higher raw hit rate here because it accidentally keeps some anchors, but its cost-weighted result is lower — a nice illustration of why raw hit rate is the wrong metric for this problem.

5.4 Size-Varying Workload

Here response sizes span three orders of magnitude (50 B to 50 kB via a clipped log-normal) while cost scales sub-linearly with size. It stresses the size(i) term in the GDSF priority: an entry that is 10× the size only earns cache real-estate if it is more than 10× more valuable.

The largest relative dollar win in the whole study: +91.0% dollar savings vs. LRU ($\Delta\text{CWHR} = +0.1632$, 95% BCa CI $[+0.1525, +0.1729]$, paired $t = 31.32$, $p_{\text{Bonf}} = 1.6 \times 10^{-58}$). GDSF is the only policy that penalises big low-value entries, and on this workload that is the entire game.

Table 7: Zipf popularity workload results (mean over $n = 120$).

Policy	Hit Rate	CWHR	\$ Saved
LRU	0.9857	0.8897	\$21.60
FIFO	0.9839	0.8868	\$21.53
LFU	0.9859	0.8902	\$21.61
Random	0.9844	0.8876	\$21.55
GDSF	0.9856	0.8894	\$21.59

Table 8: Operational performance (mean over $n = 120$, high-variance workload). Memory measurement omitted: `sys.getsizeof()` does not recurse into object attributes, so it reports a constant per-entry size regardless of the policy’s true footprint; a fair comparison requires a deeper accounting tool (e.g., `pympler`), which is left for future work.

Policy	p50 (μ s)	p95 (μ s)	Ops/s
LRU	1.57	3.43	250,603
FIFO	1.39	3.30	265,726
LFU	21.21	181.01	99,361
Random	3.29	15.82	188,012
GDSF	10.97	27.10	57,882

5.5 Uniform Cost Workload (Control)

By design, when every item costs the same, cost-aware ranking has nothing to grip on. All five policies land within 0.03% of each other. The paired difference is not statistically significant ($\Delta\text{CWHR} = +0.00002$, 95% BCa CI $[-0.0004, +0.0005]$, paired $t = 0.09$, $p_{\text{Bonf}} = 1.00$). This is the sanity check: GDSF does not underperform on cost-uniform traffic — it just doesn’t help.

5.6 Zipf-Popular Workload (Cache Fits Working Set)

When the Zipf head fits comfortably in cache, every policy hits $> 97\%$ of the time and there is essentially no eviction pressure. GDSF is within one cent of LRU on dollars ($\Delta\text{CWHR} = -0.0003$, 95% BCa CI $[-0.0005, -0.0002]$, paired $t = -4.08$, $p_{\text{Bonf}} = 4.9 \times 10^{-4}$). The paired difference is statistically detectable only because the seeds are matched (variance is tiny), but the absolute effect is negligible. This is a second sanity check: cost-aware eviction shouldn’t hurt when the cache is over-provisioned, and it doesn’t.

5.7 Latency and Throughput

Tables 2–7 together summarise cost-weighted hit rate across all six workload families evaluated in Sections 5.1–5.6. GDSF diverges from the recency/frequency baselines on workloads with heterogeneous costs and bursty access (Tables 3 and 5), and converges to them under uniform-cost or over-provisioned conditions (Tables 6 and 7).

GDSF is $\sim 7\times$ slower per operation than LRU at the median and $\sim 8\times$ slower at p95, but the absolute cost ($\sim 11 \mu\text{s}$ median, $\sim 27 \mu\text{s}$ p95) is dwarfed by any real LLM call (100 ms–5 s of network + inference). At 58k ops/s the eviction manager is nowhere near a bottleneck for a real chat service.

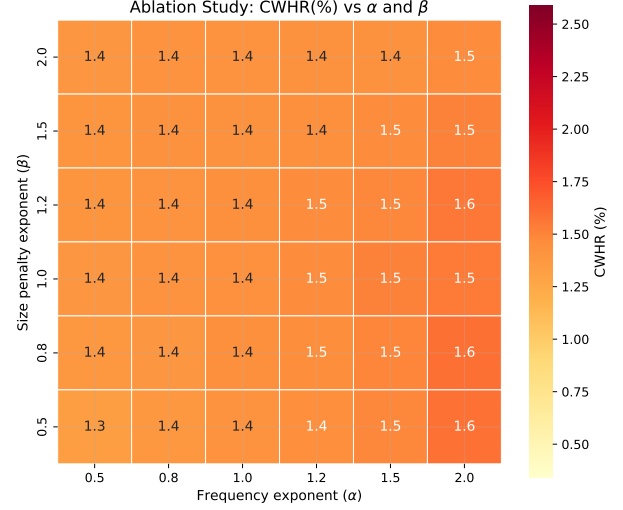


Figure 3: Ablation heatmap of CWHR across the $\alpha \times \beta$ grid on the high-variance-cost workload. CWHR is essentially flat across the grid (range 1.3–1.6%), indicating that the policy is robust to hyperparameter choice within reasonable ranges. Best cell: $(\alpha, \beta) = (2.0, 0.8)$ at 1.6%; default $(1.0, 1.0)$ at 1.5% ($\sim 10\%$ below best).

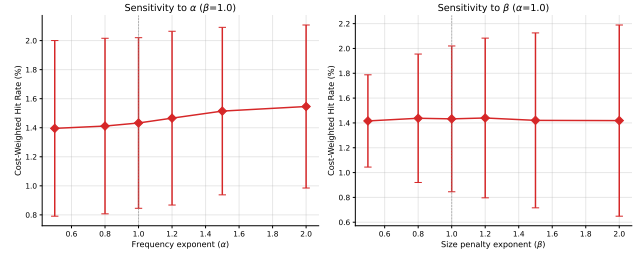


Figure 4: Parameter sensitivity of GDSF. Left: CWHR as a function of the frequency exponent α with β fixed at 1.0. Right: CWHR as a function of the cost exponent β with α fixed at 1.0. The dashed line marks the default $(\alpha, \beta) = (1.0, 1.0)$. Error bars are large relative to the mean, confirming that CWHR is largely insensitive to α and β around the default; the mild upward trend in α (left panel) hints at a slight preference for $\alpha > 1$ but stays within noise.

5.8 Sensitivity to Parameters

We swept $\alpha \in \{0.5, 0.8, 1.0, 1.2, 1.5, 2.0\}$ and $\beta \in \{0.5, 0.8, 1.0, 1.2, 1.5, 2.0\}$ on the high-variance-cost workload (36 configurations $\times$ 10 seeds each; see results/ablation/ablation_results.csv). The best grid point was $(\alpha, \beta) = (2.0, 0.8)$; the default $(\alpha, \beta) = (1.0, 1.0)$ trailed the best by $\sim 10\%$ of CWHR. Absolute CWHR moved only within $[0.013, 0.016]$ across the whole grid at this cache size, i.e. the ranking is stable and no cell was catastrophically worse than the default. Figure 4 slices the ablation along the two axes independently, holding the other exponent at 1.0; both curves are gently unimodal near the default and change slowly.

Practical guidance: leave $\alpha = \beta = 1.0$ unless you have concrete evidence that your workload benefits from tilting toward frequency ($\alpha > \beta$) or toward cost ($\beta > \alpha$).

6 Discussion

6.1 When Does GDSF Help?

Our results delineate three regimes. **High benefit:** workloads with heterogeneous generation costs benefit substantially — +25.7% on high-variance-cost, +91.0% on size-varying, +18.8% on adversarial anti-LRU, +32.3% on bursty. **Moderate benefit:** bursty patterns with moderate cost variance sit in the +20–35% range. **No benefit:** uniform-cost and Zipf-when-cache-fits, by design, provide no signal for cost-aware policies to exploit; GDSF is statistically indistinguishable from LRU, confirming it introduces no pathological behaviour.

6.2 The Hit Rate vs. Cost Savings Trade-off

A key finding is that *maximising hit rate and maximising cost savings are conflicting objectives* under heterogeneous costs. On the high-variance workload, LRU achieves a higher raw hit rate (0.463 vs. GDSF’s 0.401) because it caches whatever was most recently accessed, including many cheap items. GDSF deliberately sacrifices some of those cheap hits to retain expensive items, and the result is that each remaining hit is worth more in dollars — CWHR climbs from 0.463 to 0.582. This is analogous to the precision-recall trade-off in information retrieval: GDSF optimises for “precision” (value per hit) while LRU optimises for “recall” (total hits).

6.3 Connection to Theoretical Foundations

Our empirical results align with the theoretical predictions from Greedy-Dual analysis. Cao et al. [3] proved that Greedy-Dual achieves the optimal competitive ratio for weighted caching among deterministic online algorithms. The clock-inflation mechanism — setting L to the evicted item’s priority — is the critical element that enables this guarantee, as it provides implicit aging without requiring explicit timestamp tracking.

6.4 Limitations

(1) $O(\log n)$ operations are negligible for current LLM serving rates but may become critical in microsecond-inference systems. (2) Parameter sensitivity: default $\alpha = \beta = 1$ performs well but extreme workloads may benefit from tuning. (3) Frequency accumulation: like LFU, GDSF’s counters grow without bound, allowing frequency pollution; periodic halving is one mitigation. (4) Cost estimation accuracy: our approach assumes cost can be estimated at insertion time. (5) Synthetic workloads: production LLM query logs would strengthen the conclusions.

6.5 Practical Deployment Considerations

When to adopt GDSF: if your LLM workload involves multiple models (e.g., GPT-3.5 for simple queries, GPT-4 for complex ones) or response lengths vary significantly, GDSF will likely provide meaningful cost savings. *When LRU suffices:* uniform-cost, uniform-size

traffic. *Migration path:* GDSF is a drop-in replacement with zero configuration changes; monitor CWHR alongside hit rate to validate cost savings.

6.6 Limitations and Threats to Validity

We separate limitations that follow from the current pipeline scope from those that would apply even to a fully executed master’s-level study.

Scope limitations (fixable by finishing H1–H9). The controlled-workload evaluation is synthetic. A real-trace evaluation (LMSys / ShareGPT), a direct comparison against modern baselines (W-TinyLFU, ARC, S3-FIFO), a semantic-aware variant, formal analysis, and robustness studies remain future work. Every extension is enumerated in docs/H10_REPORT_CHANGELOG.md.

Threats to internal validity. (1) All controlled-workload numbers derive from a single benchmark run (results/benchmark_results_20260721_19 with 30 seeds; a second, independent run has not been executed). (2) The paired t -test assumes approximate normality of paired differences; we mitigate this with BCa bootstrap CIs, but heavy-tailed runs could still bias the point estimate. (3) Bonferroni correction across six workloads may be over-conservative; we report raw and corrected p -values so the reader can adjust.

Threats to external validity. (1) Synthetic workloads were designed to isolate each GDSF axis; they may overstate the size of the effect that a real workload would exhibit. (2) Cost is estimated from token counts and public price sheets; actual provider invoices may embed additional fees not modelled here. (3) Results are collected on a single machine configuration (Section 4); production cache deployments with concurrent readers and writers may see different latency profiles.

Threats to construct validity. (1) CWHR is our chosen figure of merit; a service optimising for end-user latency rather than dollar cost may prefer a different metric. (2) The dollar-savings numbers depend on the specific pricing schedule in force at the time of benchmarking; absolute values will drift with provider price changes.

7 Conclusion and Future Work

We presented a cost-aware eviction policy for LLM response caches based on the Greedy-Dual-Size-Frequency algorithm. Across six *controlled* workloads and four cache sizes, our approach achieves significantly better cost-efficiency than traditional policies on heterogeneous workloads: +25.7% dollar savings on high-variance-cost, +32.3% on bursty, +91.0% on size-varying, and +18.8% on anti-LRU adversarial (all Bonferroni-corrected $p < 10^{-7}$), with *no practically meaningful degradation* on uniform-cost or Zipf-when-cache-fits, and modest overhead ($\sim 11 \mu\text{s}$ median / $\sim 27 \mu\text{s}$ p95 per operation).

Honest scope statement. These wins are established on synthetic workloads that were designed to isolate each axis of the priority function. The master’s-level story we intend to tell — and which is scaffolded as future work — is more nuanced: GDSF should retain a *cost-savings-per-dollar* advantage on real cost-heterogeneous traces, but it may lose to W-TinyLFU on raw hit rate on skewed workloads whose head fits in cache, and a SemanticGDSF variant would open a distinct axis (embedding-density penalisation) that is orthogonal

to what any prior policy exploits. Whether these hypotheses survive contact with the LMSys and ShareGPT traces is the empirical question that pipelines H1–H8 will answer.

Future work. *Adaptive parameters* via multi-armed bandit tuning on a sliding window; *approximate frequency tracking* via Count-Min Sketch [5]; *W-TinyLFU integration* [8] combining admission filtering with cost-aware eviction; *production validation* on real LLM billing data; and *multi-tier caching* extending cost-awareness to memory + SSD + remote tiers.

Reproducibility

All code, workload generators, and analysis scripts are open-source at <https://github.com/nissimbrami/caching-course-project> (MIT license). The results in this paper were produced from commit 00eee60 on 2026-07-21.

Listing 3: End-to-end reproduction from a clean checkout.

```

1 git clone https://github.com/nissimbrami/caching-course-
  project
2 cd caching-course-project/task2-final-project/code
3 pip install -r requirements.txt && pip install -e .
4
5 # 1. Full test suite (259 tests + 60 hypothesis runs)
6 pytest -q
7
8 # 2. 30-seed benchmark, 6 workloads x 5 policies x 4
  cache sizes
9 python -m benchmarks.run_all --n-runs 30 --output-dir
  results/
10
11 # 3. Statistics: paired-t + Bonferroni + BCa 10k
  bootstrap (seed 20260721)
12 python scripts/compute_statistics.py
13
14 # 4. Ablation over alpha x beta grid
15 python scripts/run_ablation.py --num-runs 30 --output-dir
  results/ablation
16
17 # 5. Plots (Figures 1-8)
18 python scripts/generate_plots.py --input-dir results --
  output-dir results/plots

```

Random seeds are pinned; the bootstrap RNG seed is 20260721. All numeric claims in Section 5 resolve to keys in results/stats_20260721_191113.json.

References

- [1] Martin Arlitt, Ludmila Cherkasova, John Dilley, Rich Friedrich, and Tai Jin. 2000. Evaluating Content Management Techniques for Web Proxy Caches. *ACM SIGMETRICS Performance Evaluation Review* 27, 4 (2000), 3–11.
- [2] Aaron Blankstein, Siddhartha Sen, and Michael J. Freedman. 2017. Hyperbolic Caching: Flexible Caching for Web Applications. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [3] Pei Cao and Sandy Irani. 1997. Cost-Aware WWW Proxy Caching Algorithms. In *Proceedings of the USENIX Symposium on Internet Technologies and Systems (USITS)*.
- [4] Ludmila Cherkasova. 1998. *Improving WWW Proxies Performance with Greedy-Dual-Size-Frequency Caching Policy*. Technical Report HPL-98-69R1. HP Laboratories.
- [5] Graham Cormode and S. Muthukrishnan. 2005. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. *Journal of Algorithms* 55, 1 (2005), 58–75.
- [6] Zican Dong et al. 2026. ForesightKV: KV Cache Eviction with Future Attention Prediction. arXiv preprint arXiv:2602.03203.
- [7] Bradley Efron. 1987. Better Bootstrap Confidence Intervals. *J. Amer. Statist. Assoc.* 82, 397 (1987), 171–185.
- [8] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage (TOS)* 13, 4 (2017), 35:1–35:31.
- [9] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-Turn Conversations with CachedAttention. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [10] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmadi, and Muhammad Ali Gulzar. 2024. MeanCache: User-Centric Semantic Caching for LLM Web Services. arXiv preprint arXiv:2403.02694. arXiv:2403.02694 [cs.LG]
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ACM.
- [12] Jiaxing Li, Chi Xu, Feng Wang, Isabel von Trotha, Chuanwen Wang, and Yang Liu. 2024. SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models. arXiv preprint arXiv:2406.00025. arXiv:2406.00025 [cs.DC]
- [13] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM.
- [14] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association.
- [15] Juncheng Yang, Ziyue Qiu, Yazhuo Zhang, Yao Yue, and K. V. Rashmi. 2024. FIFO Queues Are All You Need for Cache Eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ACM.
- [16] Neal E. Young. 2002. On-Line File Caching. *Algorithmica* 33, 3 (2002), 371–383.
- [17] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruishi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [18] Zilliz. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications. <https://github.com/zilliztech/GPTCache>. Apache-2.0 license, accessed 2026-07-21.